

物理モデルの自動構築：データベースの拡大・層化分割・ テストケースの再生成と評価

作成者：M2 一洋

1 前回ゼミ（2026-05-19）の到達点と宿題

研究の目的：科学文献から数式を集めておき、「どんな物理モデルを作りたいか」（説明文と入出力変数）を入力すると、そのモデルを構成するのに必要な数式の集合を、まず文章の類似度で候補を絞り、次に数式どうしの変数の共有関係から作ったグラフ特徴量を多層パーセプトロン（MLP、単純な順伝播型ニューラルネットワーク）で評価して順位付けし、提示する。前回ゼミまでの到達点は次のとおりである。

- 科学論文 39 報の PDF を大規模言語モデル（Gemini 2.5 Pro）に読ませて数式を抽出し、近隣分野の基礎式も同じ大規模言語モデルに生成させて、**3,244 式・訓練ケース 1,338 件**を構築した。
- 本手法（グラフ特徴量 + MLP）と、外部手法である大規模言語モデル（LLM）による直接生成、および文章類似度だけの古典的検索を比較し、複数文献にまたがるケースで本手法が最も良い結果を示した。
- 宿題（先生方からのご指摘）：先生方から次の 3 点をご指摘いただいた。(1) 関連性の低い数式まで広く集めることが、本当に予測精度の向上に効くのかを実験で確かめること。(2) 論文 PDF をどのように収集したのかを明記すること。(3) 数式が少なく代数方程式（微分を含まない式）に偏ったテストケースの作り方を是正すること。

2 今回の進捗（要約）

前回の宿題に沿って、データの収集・分割・評価を一通りやり直した。要点は次の 6 つである。

1. データベースの拡大：分野ごとに論文を新たに収集し、収録数式を 3,244 → **11,146 式**、訓練ケースを 1,338 → **2,838 件**に増やした (§3)。
2. 訓練／テストの層化分割：数式数・入出力変数数・ソース数の分布が訓練側とテスト側でそろるように分割した。検証用データは訓練側から別に取り分け、分割を変えて 10 回繰り返し評価した (§4)。
3. テストケースの再生成：前のケースは「数式が最大 4 式と少なく、代数方程式が半分を占める」という偏りがあった。そこで加藤先生にご指摘いただいた手順でケースを作り直した (§5)。
4. API による再現可能な実装：論文用の実験はデスクトップ／Web アプリではなく API 経由のスクリプトで行えるようにした (§6)。
5. データ拡張の影響を除いた評価と特性別分析：水増し用のケースを除いた実験と、最難の DAE のみの実験を行い、数式数・ソース数・入力変数数で性能がどう変わるかを調べた (§8)。
6. 評価指標の見直し：Recall@3 を、正解式数 K に窓幅を合わせた **Recall@ K** に変えた (§7)。

3 データベースの拡大

分野ごとに論文を新たに収集し、数式データベースを拡大した。数式とケースの作り方は 3 通りである。(1) 論文 PDF を大規模言語モデル (Gemini 2.5 Pro) に読ませ、本文中の数式を抽出した (論文 329 報)。(2) 各工学分野の基礎式は、同じ大規模言語モデルに生成させた (32 分野・670 式)。(3) 複数の文献にまたがるケースは、変数を共有する数式の組を規則ベースで機械的に組み立てて作成した (大規模言語モデルは使わない)。これにより収録数式を **3,244** → **11,146** 式 (計 361 ソース)、訓練ケースを **1,338** → **2,838** 件に拡大した。ケースの内訳と、正解式数 (= 問題の難しさ) の分布を図 1 に示す。全 2,838 件のうち正解に 2 式以上を要する複数式ケースが **74.6%** を占め、複数の式を組み合わせる必要があるケースが多い。これが本手法の利点が出やすい状況である。

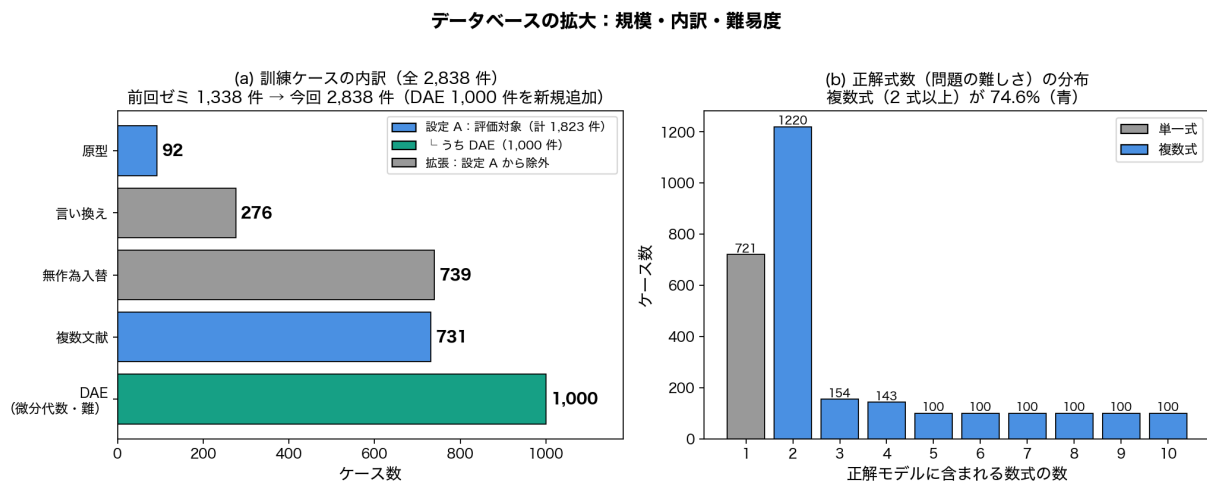


図 1 (a) 訓練ケース 2,838 件の内訳 (DAE 難ケース 1,000 件を新規追加)。評価の設定 A (§8) は原型・複数文献・DAE の 1,823 件 (92 + 731 + 1,000)。(b) 正解式数の分布：複数式 (2 式以上、青) が 74.6%。

4 訓練／テストの層化分割

評価が分割の偏りに左右されないよう、次の 2 点を満たす分割を用いた。

- ソース (文献) 単位で分ける：同じ文献から作ったケースを訓練とテストに分けて入れると、言い換えなどで答えが漏れる。これを防ぐため、文献ごと丸ごと訓練・検証・テストのいずれかに割り当てた (おおよそ 6:2:2)。検証用は訓練から取り分け、ハイパーパラメータ調整に使う。
- 特徴量の分布をそろえる：ソースをランダムに割り当てる候補を 400 通り作り、数式数・入力変数数・出力変数数・横断ソース数 (そのケースの正解式が何件の文献にまたがるか) の 4 つの分布が訓練とテストで最も近くなる候補を選んだ。

図 2 は、選ばれた分割で訓練 (青) とテスト (橙) の 4 分布がそろっている様子を示す。ランダムなソース分割 (20 通りの平均) と比べ、分布のずれ (割合の差の総和、 L_1 距離) は正解式数で $0.27 \rightarrow 0.07$ 、出力変数数で $0.27 \rightarrow 0.07$ 、横断ソース数で $0.28 \rightarrow 0.05$ と大きく縮んだ。入力変数数は値の範囲が広く連続的なため縮みは小さい ($0.44 \rightarrow 0.33$)。さらに、分割の乱数を変えた 10 通りで繰り返し評価することで、特定の分割に偏らない平均的な性能を見た (交差検証に相当)。

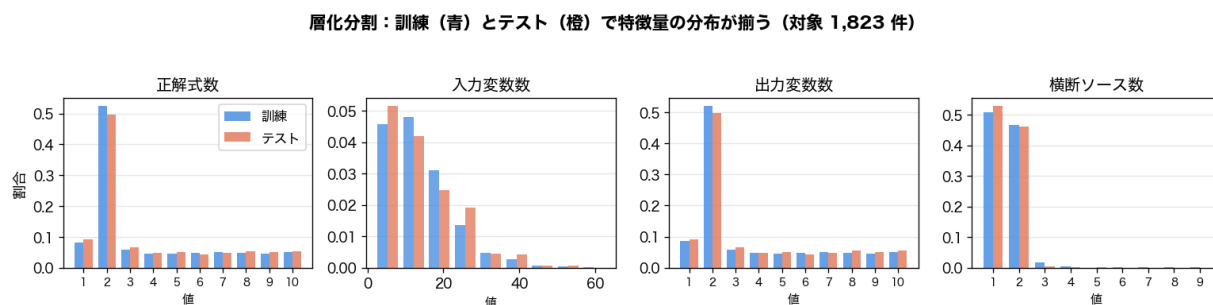


図2 層化分割：訓練（青）とテスト（橙）で4つの特徴量の分布がそろそろ（対象 1,823 件、乱数 1 例）。

5 テストケースの再生成（DAE）

前のテストケースを調べると、物理モデルを構成する数式が最大 4 式と少なく、代数方程式（時間微分を含まず、変数どうしの関係だけを表す等式。例：物質収支 $F_{\text{in}} = F_{\text{out}}$ や状態方程式 $PV = nRT$ ）が半数を占めるという偏りがあった。これでは物理モデルとして本来重要な「時間変化を表す常微分方程式（ODE）と、それに付随する代数方程式が連立する系（微分代数方程式系、DAE）」が十分に含まれない。そこで、加藤先生にご指摘いただいた次の手順でケースを作り直した（正解式数 $X = 1, \dots, 10$ について各 100 件、計 1,000 件）。

0. X を決める ($X \in \{1, 2, \dots, 10\}$)。
1. いずれかのソースから微分方程式を見つけ、正解数式集合に加える。
2. 微分方程式の左辺の微分されている変数を出力とする。
3. 正解数式集合に含まれる「出力変数以外」の変数を含む数式を、同じソースか別のソースから見つけて加える。パラメータに代入するだけの数式（左辺が変数、右辺が数値）でもよい。
4. 数式の数が X になるまで 3 を繰り返す。 X が大きいとき無限ループにならないよう、全ソースを探索したら打ち切る。
5. 正解数式集合に含まれる変数のうち、出力変数以外を入力変数とする。
6. (正解数式集合、出力変数、入力変数) の組を正解とし、これからモデルの説明文を作成する。
7. 1-6 を、正解ケースが 100 件になるまで繰り返す。

たとえば、連続槽型反応器（CSTR；反応物を連続的に供給・排出しながら攪拌する槽）で槽内の反応物濃度 C_A (mol/m³) の時間変化を求めるモデルは、次の 3 式を連立して初めて解ける。(i) 物質収支の常微分方程式 $V dC_A/dt = F(C_{A0} - C_A) - V r_A$ (V : 槽体積 m³、 F : 流量 m³/s、 C_{A0} : 入口濃度 mol/m³、 r_A : 反応速度 mol/(m³·s))、(ii) 反応速度式 $r_A = k C_A$ (k : 反応速度定数 1/s)、(iii) 反応速度定数の温度依存を表すアレニウスの式 $k = A \exp(-E/RT)$ 。手順 1 で (i) の ODE を出発点（種）とし、手順 3 で (ii)(iii) のように変数を共有する式（ここでは r_A や k ）を順に足していく。

この手順は `generate_dae_cases.py` に実装した。手順 6 の説明文生成のみ大規模言語モデル（Anthropic Claude）を用い、それ以外（ODE の探索・微分変数の抽出・変数を共有する式の追加）はすべて規則ベースで行う。生成時には、未知数（出力変数）の個数を方程式の本数 X に一致させ（一致しないケースは捨てる）、残りの変数はすべて既知の入力変数とする。未知数と式数が等しい正方系なので、入力を与えれば解が一意に定まる（これを自由度ゼロと呼ぶ）。こうして構成上解ける DAE ケースが得られる。

6 API による再現可能な実装

論文に載せる実験は、誰が再実行しても同じ結果になる必要がある。デスクトップ／Web アプリ（Claude や Gemini のアプリ）は、過去の対話ログを参照したり、内部の処理が予告なく変わったりするため、実験には使わない。本研究では LLM を使う部分（DAE ケースの説明文生成、および外部比較に用いる LLM 直接生成）を、すべて **API** を呼ぶスクリプトとして実装した（`generate_dae_cases.py`、`evaluate_llm_dae.py`；モデルは日付指定版 `claude-sonnet-4-5-20250929`）。本手法（reranker）自体は LLM を使わない決定的な Python スクリプトであり、入力（説明文・入出力変数）を与えれば第三者が同じ結果を再現できる。

7 評価指標：Recall@3 から Recall@ K へ

前回までは **Recall@3**（上位 3 件のうち正解式が占める割合）を主指標にしていた。しかし窓幅を 3 に固定すると、正解式が 4 件あるケースでは上位 3 件をすべて当てても 3/4 までは取れず、逆に正解式が 1 件のケースでは無関係な式を 2 件混ぜても満点になる。そこで、窓幅をそのケースの正解式数 K に合わせる **Recall@ K** に変えた。Recall@ K は「必要な式を、過不足ない件数だけ上位にそろえられたか」を、正解式数の多少によらず公平に測れる。たとえば Recall@ $K = 0.74$ は、必要な式のうち平均 74% を上位 K 件に正しく並べられたことを意味する。補助指標として MAP（平均適合率：正解の式をどれだけ上位にまとめられたかを表す指標）と、人手レビューを前提とした実用指標 Recall@20（上位 20 件に必要な式が入る割合）も併記する。

8 評価

8.1 実験設定（データ拡張の影響を除く）

データ拡張で水増ししたケース（単一文献の入出力を無作為に入れ替えた無作為入替ケースと、説明文を言い換えた言い換えケース）を含めると、問題本来の難しさが薄まる。そこで影響を切り分けるため、次の 2 設定で評価した。いずれも §4 の層化分割・10 通りの乱数で評価する。

- 設定 **A**：拡張を除いた原型・複数文献・DAE のみ、計 **1,823** 件。
- 設定 **B**：最も難しい DAE のみ、計 **1,000** 件。

拡張を除く効果は数値で確かめられる（図 3）。全 2,838 件（拡張込み）で同じ手順で評価すると、baseline の Recall@ K は 0.622 となり、拡張を除いた設定 **A**（0.521）より 0.101 も高く出る。易しい拡張ケースが baseline を過大に見せるため、本手法との差は設定 **A** の +0.222 から、拡張込みでは +0.156 に縮んで見える。すなわち拡張を残したまま評価すると、提案手法の効果を約 0.07 過小評価してしまう。これが拡張を除く理由である。

比較対象は、**baseline**（古典的情報検索：説明文と数式の文章類似度などで並べる第 1 段。特徴量は文章・変数の類似度など 7 個）と、**reranker-10S**（本手法：第 1 段の候補を、数式どうしの変数共有関係から作った集合特徴量も加えて並べ直す第 2 段）の 2 つである。ここで集合特徴量とは、各候補式を単独でなくすでに選んだ式の集合との関係で評価する量で、未充足の入力変数をどれだけ補えるか（補完性）・既選択集合と変数をどれだけ共有するか（一貫性）・同じ分野か（ドメイン一致）の

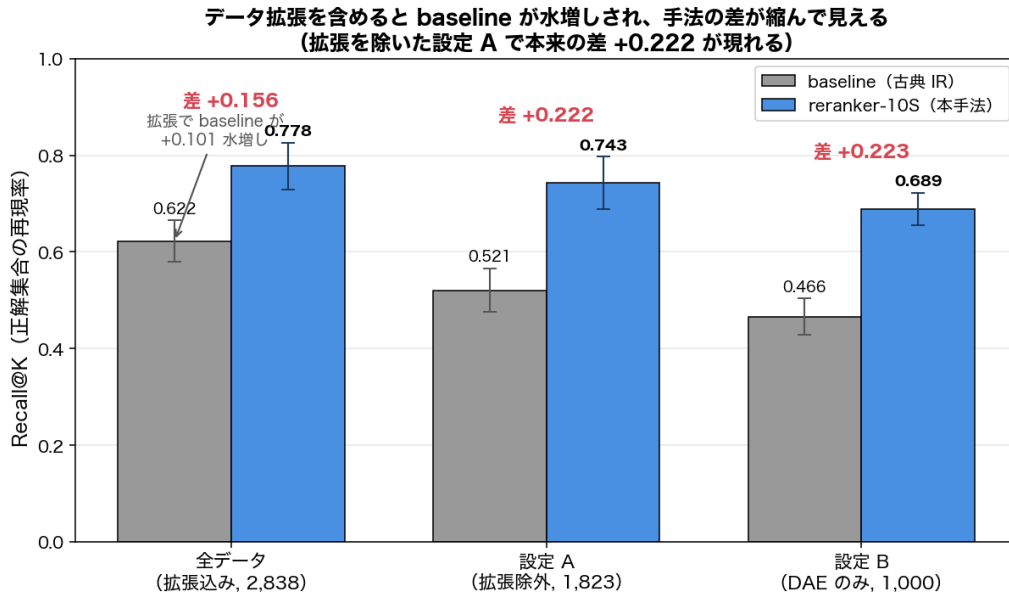


図3 データ拡張の影響：全データ（拡張込み）・設定 A（拡張除外）・設定 B（DAE のみ）での baseline と本手法の Recall@K（10 乱数平均、エラーバーは乱数間の標準偏差）。易しい拡張ケースは baseline を +0.101 水増しし、本手法との差を +0.222 → +0.156 に縮めて見せる。拡張を除いた設定 A で本来の差が現れる。

3 個からなる。名称の「10S」は、第 2 段で使う特徴量が計 10 個（基本 7 個＋集合特徴量〔英語で set-aware〕3 個）であることを表す。並べ直しには多層パーセプトロン（MLP）を用いる。

8.2 全体の結果

表 1 に両設定の結果を示す。本手法は両設定とも Recall@K を +0.22 ポイント引き上げ、必要な式集合のうち上位 K 件に正しくそろう割合を、設定 A で 0.521 → 0.743 (+0.222)、最難の設定 B でも 0.466 → 0.689 (+0.223) に改善した。人手レビュー前提の Recall@20 は設定 A で 0.91、DAE のみでも 0.84 に達する。文章類似度だけの baseline では、複数の文献にまたがる式や変数の共有関係を捉えきれないことが、差にそのまま表れている。

表 1 層化分割・10 通りの乱数での平均（± は乱数間の標準偏差）。本手法は両設定で Recall@K を +0.222（設定 A）・+0.223（設定 B）改善する。

設定	手法	Recall@K	MAP	Recall@20
A（原型＋複数文献＋ DAE, 1,823 件）	baseline	0.521 ± 0.045	0.581	0.836
	reranker-10S（本手法）	0.743 ± 0.054	0.863	0.909
B（DAE のみ, 1,000 件）	baseline	0.466 ± 0.037	0.512	0.721
	reranker-10S（本手法）	0.689 ± 0.033	0.832	0.838

8.3 特性格の分析：どんなケースで性能が変わるか

設定 A のケースごとの結果を、数式数・ソース数・入力変数数で層別した（図 4）。

- 数式数（図 a）：数式が多いほど難しくなる。baseline は正解式数が 1 のとき 0.85 だが 10 では

0.28 まで落ちる。本手法も $0.88 \rightarrow 0.54$ と下がるが落ち方は緩やかで、差は単一式の $+0.03$ から複数式の $+0.21 \sim +0.29$ へ広がる。「易しい問題を壊さず、難しい組合せ問題で効く」という望ましい性質を示す。

- 入力変数数 (図 c) : 入力変数が多いほど難しくなる。baseline は 5 個以下の 0.80 から 21 個以上の 0.37 へ落ちる一方、本手法は $0.85 \rightarrow 0.66$ にとどまり、差は $+0.05$ から $+0.28$ へ広がる。
- ソース数 (図 b) : ソース数そのものは性能とほぼ無関係だった。これはソース数が数式数と交絡している (2 文献のケースの多くは 2 式の易しい問題) ため、本手法の優位はソース数によらず $+0.22 \sim +0.26$ でほぼ一定である。

まとめると、数式数と入力変数数が増えるほど問題は難しくなり、そこでこそ本手法の上乗せが大きくなる。出力変数数は正解式数とほぼ一致するため、数式数と同じ傾向を示す。

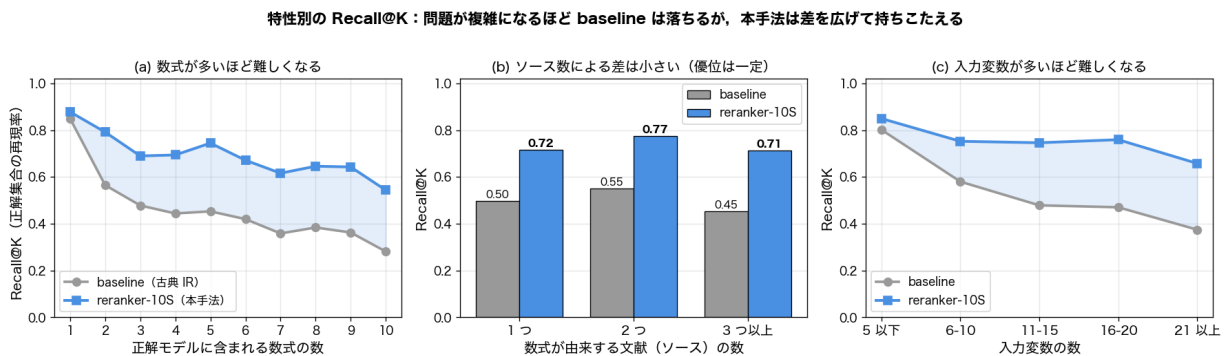


図 4 特性格の Recall@K (設定 A、ケース単位を 10 乱数で平均)。(a) 数式数、(b) ソース数、(c) 入力変数数。問題が複雑になるほど baseline (灰) は落ちるが、本手法 (青) は差を広げて持ちこたえる。

8.4 留意点

本結果には次の前提がある。(1) DAE ケースの説明文は LLM が生成しており、表現の揺れを含む。(2) ソース数の効果は数式数と交絡しているため、単独の因果とは解釈できない。(3) 値は 10 通りの乱数分割の平均であり、層化分割は 400 候補から選んだ 1 つの方式である。

9 まとめと今後

今回：データベースを 11,146 式・2,838 件に拡大し、分布をそろえた層化分割と、加藤先生ご指摘の手順による DAE ケースの再生成を行い、API で再現可能な実験基盤を整えた。評価指標を Recall@K に改めたうえで、データ拡張の影響を除いた 2 設定で評価した結果、本手法は Recall@K を $+0.22$ ポイント改善 (設定 A $+0.222$ 、設定 B $+0.223$) し、その効果は数式数・入力変数数が多い難しいケースほど大きいことを特性格分析で確かめた。

今後：

- PSE Asia 発表の準備 (本報告のデータ拡大・層化分割・DAE 再生成・特性格評価を中核に据える)。
- 数式数・入力変数数が多いケースで性能が下がる問題に対し、手法を改善する。
- 論文執筆 (API ベースの再現可能な実験一式を整備し、LLM 直接生成との同条件比較を DAE ま

で拡張する)。